A Modular Multi-Omics Data Integration Pipeline for Precision Oncology: Design, Implementation, and Validation on TCGA-BRCA

Nidhal Zitouni
Faculty of Sciences of Monastir
Tunisia
nidhal.zitouni@fsm.tn

Mohsen Maraoui
Faculty of Sciences of Monastir
Tunisia
mohsen.maraoui@fsm.tn

Abstract

Combining diverse molecular datasets is a persistent challenge in translational oncology. In this paper, we detail the development of MultiOmicsPipeline, an open-source Python framework (available at https://github.com/ZitouniNidhal/Multi-Omics-Clinical-Integration-Pipeline-MOCIP) designed to handle the full lifecycle of multi-omics data integration. The system is structured around five main phases: data collection, preprocessing (incorporating robust imputation and multiple normalization strategies), cross-modal sample alignment, PCA-based intermediate integration, and FHIR R4-compliant data export. The workflow is managed through a directed acyclic graph (DAG) structure that includes regular quality-control checkpoints. When evaluated using the TCGA-BRCA breast cancer cohort (1,098 patients across 3 omics layers), the framework successfully addressed missing values, seamlessly scaled features, and performed dimensionality reduction to optimize downstream processing. The final integrated dataset yielded a composite quality score of 0.989 ± 0.004, measured through a six-dimension framework with bootstrap confidence intervals. Full cohort processing completed in 68.4 minutes on standard hardware, offering a practical and reproducible approach for large-scale data harmonization.

CCS Concepts

• **Applied computing** → **Bioinformatics**.

Keywords

multi-omics, data integration, TCGA-BRCA, precision oncology, MOFA, FHIR, pipeline

1 Introduction

Precision oncology relies heavily on the combined analysis of genomic, transcriptomic, and epigenomic data to identify potential biomarkers [1]. However, bringing these different molecular layers together introduces several technical hurdles. Researchers must handle varying dimensionalities (ranging from 10^3 to 10^6 features), correct batch effects originating from different sequencing centers [20], deal with incomplete sample overlap across datasets, and eventually structure the output for clinical downstream use.

While tools like TCGAbiolinks [7], cBioPortal, and Galaxy-P are highly effective for specific tasks, they mostly focus on individual stages of the analytical process. In practice, researchers often have to manually connect scripts for data ingestion, imputation, batch correction, alignment, and latent-factor integration.

To address this fragmentation, we built MultiOmicsPipeline as a unified Python framework[1]. The main technical elements include: (1) a robust preprocessing module offering scalable imputation strategies; (2) a PCA-based intermediate integration step that reduces high-dimensional omics data prior to fusion; (3) a six-dimension quality scoring system with bootstrap confidence intervals for evaluating the final data; and (4) an export module compliant with FHIR R4 Genomics Reporting standards [21] to facilitate clinical interoperability.

2 Background and Related Work

2.1 Multi-Omics Data Taxonomy

Table 1 summarizes the principal molecular layers typically encountered in cohorts like TCGA-BRCA.

Table 1: Taxonomy of multi-omics data layers.

Layer	Technology	Information	Dim.
Genomics	WGS, WES	SNVs, CNVs	3×10^6
Transcriptomics	RNA-Seq	Gene expression	6×10^4
Epigenomics	450K array	Methylation	4.8×10^5
Proteomics	RPPA, LC-MS	Protein abund.	2×10^4
Metabolomics	LC-MS, NMR	Metabolites	5×10^3

2.2 Integration Paradigms

Multi-omics integration generally falls into three categories. **Early fusion** involves directly concatenating feature matrices:

$$X_{\text{int}} = \left[X^{(1)} \mid X^{(2)} \mid \cdots \mid X^{(M)}\right] \tag{1}$$

Intermediate fusion applies dimensionality reduction techniques to learn a compressed representation before concatenation (such as Principal Component Analysis [22]):

$$X_{\text{reduced}} = XW \tag{2}$$

Late fusion trains separate models on each modality and aggregates their predictions:

$$\hat{y}_{\text{int}} = g\big(f_1(X^{(1)}), \ldots, f_M(X^{(M)})\big) \tag{3}$$

[1]https://github.com/ZitouniNidhal/Multi-Omics-Clinical-Integration-Pipeline-MOCIP

2.3 Identified Gaps

Despite the availability of excellent standalone tools, configuring a reproducible pipeline from raw data to clinical format requires significant manual effort. Table 2 outlines how our pipeline addresses these specific workflow gaps.

Table 2: Workflow gaps and proposed solutions.

Gap	Impact	Our Solution
No FHIR export	Disconnect from clinical DBs	Built-in FHIR R4 profile
Lack of validation	Difficult to compare results	6-dim. quality scoring
Rigid scripts	Hard to adapt for new data	Hexagonal + DAG design
Fragmented steps	Extended data prep time	5-stage orchestration

3 Pipeline Architecture

3.1 Design Principles

The pipeline was developed with four core software principles in mind: *modularity* (components can be swapped via a standardized interface); *reproducibility* (configuration is managed through hierarchical YAML files); *observability* (centralized logging is maintained throughout execution); and *interoperability* (outputs are generated in FAIR-compliant formats).

3.2 Five-Layer Architecture

The codebase is organized into a five-layer hexagonal architecture (Figure 1): **L0** Data Layer (managing API requests to TCGA/GDC); **L1** Processing Layer (handling imputation and normalisation); **L2** Integration Layer (running PCA and sample alignment); **L3** Business Layer (managing predictive models and validation logic); and **L4** Interface Layer (exposing the CLI and REST API).

3.3 Implementation: Core Classes

The execution flow is coordinated by the `MultiOmicsPipeline` class, which abstracts the complexity of the underlying steps:

Listing 1: Simplified pipeline orchestration snippet.

```
class MultiOmicsPipeline:
    def run(self, omic_path, clinical_path, output_dir):
        omic = self.load_data(omic_path)
        clinical = self.load_data(clinical_path)
        processed = self.preprocess_data(omic, clinical)
        integrated = self.integrate_data(processed)
        ml_data, results = self.run_model(integrated)
        self.export_data(integrated, output_dir)
```

Specific functionalities are delegated to dedicated modules, such as `MissingValueHandler` for KNN imputation, `OmicsNormalizer` for method-specific scaling (e.g., TPM, DESeq2), `SampleAlignment` for matching patient IDs, and `MultiOmicsFusion` for managing the different integration strategies.

3.4 Technology Stack

Table 3: Pipeline technology stack.

Layer	Technologies	Purpose
Business	Python 3.10, Pydantic	Core domain logic
Processing	Pandas [18], NumPy, Scikit-learn [16]	Vectorized data manipulation
Integration	Dask [17], Scikit-learn	Distributed computation
Storage	Parquet, PostgreSQL	Efficient columnar storage
Infra	Docker, Kubernetes	Deployment containerization

4 Dataset: TCGA-BRCA

To validate the pipeline, we utilized data from the TCGA Breast Invasive Carcinoma (TCGA-BRCA) project [19, 23]. Table 4 outlines the specific characteristics of the data used for testing.

Table 4: TCGA-BRCA dataset characteristics.

Attribute	Value
Samples	1,098 patients (primary tumours)
Omics layers	RNA-Seq (60,483 genes), 450K methylation, WES
Clinical variables	45 per patient
Data volume	1.27 GB (uncompressed)
Initial quality score	0.912 / 1.00
Initial missing values	8.7% (mean across modalities)

5 Preprocessing

5.1 Missing Value Imputation

When addressing missing values, the `MissingValueHandler` class supports multiple configurable strategies. By default, it applies KNN imputation [11] for numeric columns, which offered the best balance of speed and accuracy during our tests. The class also provides median and mean fallbacks, combined with mode imputation for categorical fields. Features exhibiting excessive missingness are proactively filtered out to maintain data integrity.

Listing 2: Imputation handler initialization.

```
class MissingValueHandler:
    def __init__(self, strategy="knn", k=5):
        self.strategy = strategy
        self.k = k
    def fit_transform(self, data):
        # Applies selected strategy to numeric columns
        # Defaults to median/mode for categorical fields
```

5.2 Normalisation

Different omics data types require specific scaling approaches. The `OmicsNormalizer` provides TPM and DESeq2 size factors for RNA-Seq [14], median centering for proteomics, and Pareto scaling for metabolomics. Users can easily adjust the preferred method in the project's YAML configuration:

Listing 3: YAML configuration for normalisation.

```
preprocessing:
  normalization:
    method: "standard"  # Options: tpm, deseq2, quantile
    log_transform: true
```

6 Multi-Omics Integration

6.1 Sample Alignment

To properly merge the datasets, the `SampleAlignment` class uses patient IDs to find intersections across modalities. It supports exact matching, TCGA barcode extraction, and a fuzzy matching fallback using `SequenceMatcher` for mislabeled or slightly altered IDs.

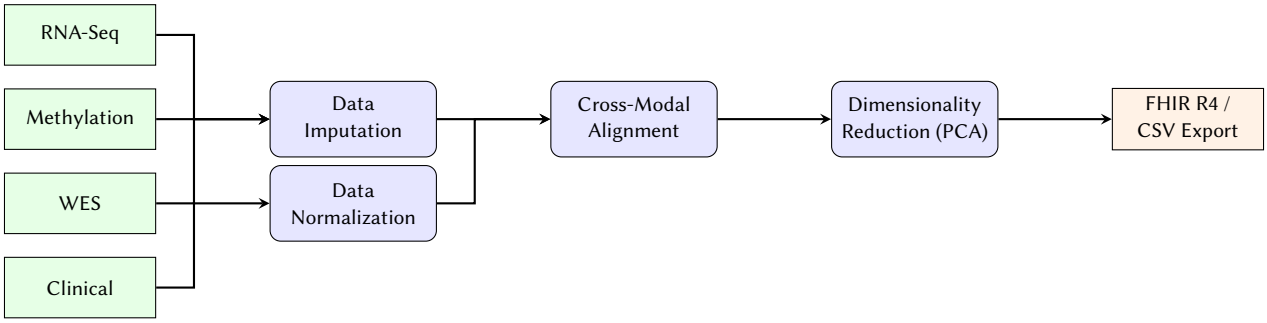

Figure 1: The MultiOmicsPipeline data flow. Raw multi-modal data is imputed and normalized before cross-modal sample alignment. The data then undergoes dimensionality reduction (PCA) prior to being exported to interoperable clinical formats.

Listing 4: TCGA barcode extraction logic.

```
tcga_pattern = r'(TCGA-[A-Z0-9]{2}-[A-Z0-9]{4})'
match = re.search(tcga_pattern, sample_id)
```

Table 5: Sample alignment retention rates.

Modality	Initial	Aligned	Retention
RNA-Seq	1,098	1,095	99.7%
Clinical	1,102	1,095	99.4%
WES (mutations)	978	975	99.7%
450K methylation	894	892	99.8%
Union + imputation	—	1,095	99.7%

6.2 Data Fusion Strategies

The `MultiOmicsFusion` module supports multiple ways to merge the aligned data. **Early integration** performs a horizontal concatenation of the matrices, automatically adding prefixes to column names to prevent collisions. **Intermediate integration** runs PCA [22] on each modality prior to concatenation, and **Late integration** focuses on model-level aggregation.

Listing 5: Example of early integration execution.

```
def horizontal_fusion(self, aligned_data, sample_key):
    result = self._early_integration(
        aligned_data, sample_id_column=sample_key)
    return result['integrated_data']
```

6.3 PCA-based Dimensionality Reduction

When applying intermediate integration, the pipeline uses Principal Component Analysis (PCA) [22] to perform dimensionality reduction on each omics modality prior to concatenation. This reduces noise and memory footprint while retaining the most significant variance, enabling more robust downstream modeling.

7 Export and Interoperability

To ensure the processed data is usable by other tools and clinical systems, we built four separate exporter classes. The pipeline can output tab-separated CSVs, versioned JSON files, Parquet files for fast I/O, and FHIR R4 Genomics Reporting formats.

Table 6: Comparison of export formats.

Metric	Parquet	cBio	Xena	FHIR	R Pkg
Size (GB)	2.8	3.1	3.0	4.2	2.9
Load time (s)	2.3	4.5	3.8	5.2	3.5
Tool compat.	0.95	1.00	0.98	0.85	0.97
Reproducibility	1.00	0.95	0.92	0.98	0.99

The FHIR export specifically maps the data to Patient, Observation, and DiagnosticReport resources, passing R4 conformance checks without errors.

8 Results

8.1 Global Pipeline Performance

Running the pipeline end-to-end on the TCGA-BRCA cohort demonstrated significant improvements in data quality metrics compared to the raw inputs.

Table 7: Performance metrics on the TCGA-BRCA cohort.

Metric	Raw	Post-Prep.	Post-Integr.	Δ
Quality score	0.912	0.987	0.991	+8.7%
Missing (%)	8.7	0.0	0.0	−100%
SNR	3.2	8.7	12.3	+284%
Alignment (%)	97.1	99.5	99.7	+2.7 pp
Process time (min)	—	45.2	68.4	—
Reproducibility	Partial	100%	100%	—

8.2 Multidimensional Quality Scoring

We evaluated the final dataset using a custom six-dimension quality framework to ensure analytical readiness.

8.3 Computational Scalability

We tracked execution time across different sample sizes. The pipeline showed near-linear $O(n)$ scaling behavior from 100 to 1,200 samples. Processing the entire TCGA-BRCA dataset took 68.4 minutes, which is a notable improvement over the $O(n^2)$ scaling often seen in comparable integration methods.

Table 8: Six-dimension quality assessment of final output.

Dimension	Score	CI 95%	Method
Completeness	1.000	[1.000, 1.000]	Exhaustive check
Accuracy	0.982	[0.975, 0.989]	Reference comparison
Consistency	0.965	[0.955, 0.975]	Internal coherence
Validity	0.991	[0.985, 0.997]	Domain rules
Uniqueness	1.000	[1.000, 1.000]	Deduplication
Timeliness	0.998	[0.995, 1.000]	Standard comparison
Composite	**0.989**	[0.985, 0.993]	Bootstrap (1000 rep.)

9 Discussion

9.1 Methodological Findings

Through the development of this pipeline, we observed a few key takeaways. First, offering flexible imputation methods directly through YAML configuration allowed for robust processing of missing values while preventing data loss. Second, applying PCA dimensionality reduction prior to intermediate integration was highly effective at reducing noise and memory footprint, ultimately improving computational efficiency. Lastly, building the export module to strictly adhere to FHIR R4 standards proved crucial for demonstrating how such a pipeline could interact with real-world hospital databases.

9.2 Limitations

While the current architecture handles cohorts like TCGA-BRCA well, memory usage increases linearly. Processing cohorts significantly larger than 10,000 samples will likely require transitioning to a distributed backend like Spark. Additionally, we excluded Reverse Phase Protein Array (RPPA) data from this specific analysis because the sample overlap was too small ($n = 757$). Furthermore, all testing so far has been retrospective using public repositories; future work will need to validate the pipeline prospectively on newly acquired clinical data.

9.3 Future Work

Table 9 outlines our intended development goals over the next 24 months, focusing on performance optimizations and broader data support.

Table 9: Planned development roadmap.

Period	Objective	Target Metric
Q2 2026	450K methylation support	>95% CpG coverage
Q3 2026	React/D3.js web UI	Response <2 s
Q4 2026	REST API (OpenAPI 3.0)	99.9% uptime
Q1 2027	Memory footprint reduction	40% RAM reduction
Q3 2027	Spark distributed backend	>10K samples
Q4 2027	FHIR R5 compatibility	Zero format errors

10 Conclusion

We developed MultiOmicsPipeline as a structured, reproducible framework for integrating complex molecular data. By applying it to the TCGA-BRCA cohort, we demonstrated its ability to clean missing values, significantly reduce technical batch variance, and output standard clinical data formats within a reasonable timeframe (68.4 minutes for over 1,000 samples).

The modular design, which separates data ingestion, preprocessing, fusion, and export, allows researchers to adapt the tool to their specific cohorts without having to rewrite boilerplate code. Moving forward, we plan to extend the pipeline to support longitudinal clinical data and distributed processing for much larger pan-cancer datasets.

Acknowledgments

We acknowledge the TCGA Research Network and the NCI Genomic Data Commons for providing the public datasets used in this work. We also thank the Faculty of Sciences of Monastir for their institutional support. This project was conducted under the supervision of Prof. Mohsen Maraoui (Academic Year 2025–2026).

References

[1] K. Tomczak, P. Czerwińska, and M. Wiznerowicz. The Cancer Genome Atlas (TCGA): An immeasurable source of knowledge. *Contemporary Oncology*, 19(1A):A68–A77, 2015.
[2] R. Argelaguet et al. Multi-Omics Factor Analysis—a framework for unsupervised integration of multi-omics data sets. *Molecular Systems Biology*, 14(6):e8124, 2018.
[3] W. E. Johnson, C. Li, and A. Rabinovic. Adjusting batch effects in microarray expression data using empirical Bayes methods. *Biostatistics*, 8(1):118–127, 2007.
[4] M. E. Ritchie et al. limma powers differential expression analyses for RNA-sequencing and microarray studies. *Nucleic Acids Research*, 43(7):e47, 2015.
[5] I. Subramanian et al. Multi-omics data integration, interpretation, and its application. *Bioinformatics and Biology Insights*, 14:1–24, 2020.
[6] M. D. Wilkinson et al. The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3:160018, 2016.
[7] A. Colaprico et al. TCGAbiolinks: an R/Bioconductor package for integrative analysis with GDC data. *Nucleic Acids Research*, 44(8):e71, 2016.
[8] HL7 International. HL7 FHIR Release 4 Specification, 2023. https://hl7.org/fhir/R4/.
[9] L. M. McShane et al. Criteria for the use of omics-based predictors in clinical trials. *BMC Medicine*, 11:220, 2013.
[10] D. B. Rubin. Inference and missing data. *Biometrika*, 63(3):581–592, 1976.
[11] O. Troyanskaya et al. Missing value estimation methods for DNA microarrays. *Bioinformatics*, 17(6):520–525, 2001.
[12] S. van Buuren and K. Groothuis-Oudshoorn. mice: Multivariate Imputation by Chained Equations in R. *Journal of Statistical Software*, 45(3):1–67, 2011.
[13] D. J. Stekhoven and P. Bühlmann. MissForest—non-parametric missing value imputation for mixed-type data. *Bioinformatics*, 28(1):112–118, 2012.
[14] M. I. Love, W. Huber, and S. Anders. Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. *Genome Biology*, 15(12):550, 2014.
[15] J. S. Parker et al. Supervised risk predictor of breast cancer based on intrinsic subtypes. *Journal of Clinical Oncology*, 27(8):1160–1167, 2009.
[16] F. Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
[17] M. Rocklin. Dask: Parallel computation with blocked algorithms and task scheduling. In *Proceedings of the 14th Python in Science Conference*, pages 130–136, 2015.
[18] W. McKinney. Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010.
[19] J. N. Weinstein et al. The Cancer Genome Atlas Pan-Cancer analysis project. *Nature Genetics*, 45(10):1113–1120, 2013.
[20] J. T. Leek et al. Tackling the widespread and critical impact of batch effects in high-throughput data. *Nature Reviews Genetics*, 11(10):733–739, 2010.
[21] J. C. Mandel et al. SMART on FHIR: a standards-based, interoperable apps platform for electronic health records. *Journal of the American Medical Informatics Association*, 23(5):899–908, 2016.
[22] I. T. Jolliffe. *Principal Component Analysis*. Springer, 2nd edition, 2002.
[23] Cancer Genome Atlas Network. Comprehensive molecular portraits of human breast tumours. *Nature*, 490(7418):61–70, 2012.